



BANNARI AMMAN INSTITUTE OF TECHNOLOGY

An Autonomous Institution Affiliated to Anna University Chennai - Approved by AICTE - Accredited by NAAC with "A+" Grade

SATHYAMANGALAM - 638 401 ERODE DISTRICT TAMIL NADU INDIA

Ph: 04295-226000 / 221289 Fax: 04295-226666 E-mail: stayahead@bitsathy.ac.in Web: www.bitsathy.ac.in

22XX404 WEB TECHNOLOGY AND FRAMEWORKS

UNIT II & LP 1– Client-side Scripting vs Server-Side Scripting

1. Client-Side Scripting:

Definition: Client-side scripting refers to scripts that are executed on the user's web browser (client). The code is downloaded along with the web page and is then processed by the user's browser.

Languages: Common client-side scripting languages include JavaScript, HTML, and CSS. JavaScript is the most widely used language for client-side scripting.

Responsibilities: Client-side scripting is primarily responsible for enhancing the user interface and interactivity of a website. It can be used for tasks such as form validation, dynamic content updates, and creating responsive user interfaces.

Advantages: Reduces the load on the server because the client's browser handles the execution. Provides a more dynamic and interactive user experience.

1.1 Server-Side Scripting:

Definition: Server-side scripting involves running scripts on the web server. The server processes the script and sends the result (usually HTML) to the client's browser, which then renders the web page.

Languages: Popular server-side scripting languages include PHP, Python (Django), Ruby (Ruby on Rails), Java (Servlets, JSP), and Node.js (JavaScript on the server).

Responsibilities: Server-side scripting is responsible for handling tasks that involve server resources, database interactions, authentication, and more. It is often used for generating dynamic content, processing form submissions, and interacting with databases.

Advantages: Provides better security for sensitive operations since the code is executed on the server and not exposed to the client. Can be used for complex business logic and server-side processing.

Combination:

In many web applications, a combination of client-side and server-side scripting is used to

achieve the desired functionality. For example, client-side scripts may handle form validation for a better user experience, while server-side scripts handle the actual processing of the form data and database interactions.

Here's a breakdown of the execution location for client-side scripting:

Client-side scripting refers to scripts that are executed on the user's web browser or client machine. The execution of client-side scripts takes place on the client side of the web application, which means the user's device is responsible for processing and running the code. The client-side scripting languages, such as JavaScript, HTML, and CSS, are downloaded along with the web page and executed by the browser.

User's Browser:

The client-side scripts are downloaded by the user's web browser when they access a web page. The browser interprets and executes the client-side code locally on the user's device.

Client Machine:

All the processing related to client-side scripting occurs on the client machine, meaning it utilizes the resources (CPU, memory) of the user's device.

User Experience:

Client-side scripting is primarily responsible for enhancing the user interface and interactivity of the website, providing a more dynamic and responsive user experience.

Advantages:

Reduces the load on the web server as the client's browser handles the execution.

Allows for interactive features and dynamic content without requiring constant communication with the server.

Examples of client-side scripting languages include JavaScript, which is commonly used for tasks such as form validation, dynamic content updates, and creating interactive user interfaces. The primary focus of client-side scripting is on improving the user experience by providing responsive and dynamic elements within a web page.

2. JavaScript Fundamentals:

JavaScript is a versatile and widely used programming language that is primarily employed for front-end web development. It allows developers to create dynamic and interactive content within web browsers. Below are some fundamental concepts and features of JavaScript:

2.1 Variables:

- Variables are used to store data values. You can declare variables using `var`, `let`, or `const`.

javascript

```
var myVar = 10;  
let myVar2 = "Hello";  
const PI = 3.14;
```

2.2 Data Types:

- JavaScript has dynamic typing, meaning variables can hold values of any data type.
- Common data types include numbers, strings, booleans, arrays, and objects.

2.3 Operators:

- JavaScript supports various operators, such as arithmetic operators (+, -, *, /), comparison operators (==, ===, !=, !==), and logical operators (&&, ||, !).

2.4 Functions:

- Functions are blocks of reusable code. They can be declared using the function keyword.

javascript

```
function greet(name) {  
    console.log("Hello, " + name + "!");  
}  
  
greet("John");
```

2.5 Control Flow:

- JavaScript supports conditional statements (if, else, switch) and loops (for, while, do-while) for controlling the flow of the program.

2.6 Arrays:

- Arrays are used to store multiple values in a single variable.

javascript

```
var fruits = ["Apple", "Banana", "Orange"];
```

2.7 Objects:

- Objects are collections of key-value pairs and are used to represent real-world entities.

javascript

```
var person = {  
  name: "John",  
  age: 30,  
  isStudent: false  
};
```

2.8 DOM Manipulation:

- JavaScript interacts with the Document Object Model (DOM) to manipulate HTML and CSS. This allows for dynamic updates to the content and style of a web page.

```
document.getElementById("myElement").innerHTML = "New Content";
```

2.9 Events:

- JavaScript can respond to user actions through events. Event handling allows you to execute specific functions when events occur (e.g., button click, form submission).

```
document.getElementById("myButton").addEventListener("click", function() {  
  alert("Button Clicked!");  
});
```

2.10 Asynchronous JavaScript:

- JavaScript supports asynchronous programming using callbacks, promises, and async/await. This is crucial for handling tasks like fetching data from servers without blocking the user interface.

2.11 JSON (JavaScript Object Notation):

- JSON is a lightweight data interchange format. JavaScript provides methods (JSON.parse())

and `JSON.stringify()` for working with JSON data.

3. Variables

JavaScript uses variables to store and manipulate data. Variables are containers for holding values, and each variable has a specific data type. Here are the basic JavaScript data types and how to work with variables:

```
javascript
```

```
var x = 5;
```

`let` (block-scoped, reassignable):

```
javascript
```

```
let message = "Hello, World!";
```

`const` (block-scoped, immutable):

```
javascript
```

```
const PI = 3.14;
```

3.1 Data Types:

JavaScript is a loosely typed language, meaning variables can hold values of any data type. The most common data types are:

1. Numbers:

javascript

```
var num = 42; // Integer  
var floatNum = 3.14; // Floating-point number
```

2. Strings:

javascript

```
var name = "John";  
var message = 'Hello, ' + name + '!';
```

3. Booleans:

javascript

```
var isTrue = true;  
var isFalse = false;
```



4. Arrays:

javascript

```
var fruits = ["Apple", "Banana", "Orange"];
```

5. Objects:

javascript

```
var person = {  
  name: "John",  
  age: 30,  
  isStudent: false  
};
```

6. Undefined:

javascript

```
var undefinedVar;
```



7. Null:

javascript

```
var nullVar = null;
```

8. Symbol (introduced in ES6):

javascript

```
var sym = Symbol("unique");
```

Checking Data Types:

You can use the `typeof` operator to check the data type of a variable:

javascript

 Copy code

```
console.log(typeof num);      // "number"
console.log(typeof name);     // "string"
console.log(typeof isTrue);   // "boolean"
console.log(typeof fruits);   // "object" (Arrays are a type of object)
console.log(typeof person);   // "object"
console.log(typeof undefinedVar); // "undefined"
console.log(typeof nullVar);   // "object" (typeof null is a known
console.log(typeof sym);      // "symbol"
```

5. Interaction: alert, prompt, confirm

In JavaScript, the `alert`, `prompt`, and `confirm` functions are used to interact with users through dialog boxes in a web browser. These functions are part of the `Window` object and are commonly employed to gather input, display messages, or confirm user actions.

1. `alert()` Function:

- The `alert()` function is used to display a simple message in an alert dialog box.

javascript

 Copy code

```
alert("Hello, World!");
```

2. `prompt()` Function:

- The `prompt()` function is used to display a dialog box that prompts the user for input. It takes two arguments: the message to display and an optional default value.
- The function returns the user's input as a string or `null` if the user clicks the "Cancel" button.

javascript

 Copy code

```
var userInput = prompt("Please enter your name:", "John Doe");
```

3. `confirm()` Function:

- The `confirm()` function is used to display a dialog box with a message and OK and Cancel buttons. It returns a boolean value (`true` if the user clicks OK, `false` if the user clicks Cancel).

javascript

 Copy code

```
var userConfirmed = confirm("Are you sure you want to proceed?");
```

6. Type Conversions:

Type conversion in JavaScript refers to the process of converting a value from one data type to another. JavaScript is a dynamically-typed language, which means that variables can hold values of any data type, and the type of a variable can be changed during the execution of the program. There are two types of type conversion: implicit (automatic) and explicit (manual).

6.1 Implicit Type Conversion:

This occurs automatically when values are mixed in operations or expressions that involve different data types. JavaScript will try to convert one or more values to a common data type before performing the operation.

```
javascript Copy code  
  
var numString = "42";  
var num = 10;  
  
var result = numString + num; // Implicit conversion to string  
console.log(result); // "4210"
```

In the example above, the number 10 is implicitly converted to a string to perform string concatenation.

6.2 Explicit Type Conversion:

Developers can explicitly convert values from one data type to another using built-in functions or methods. This is also known as type casting.

1. String Conversion:

- `String(value)` can be used to convert a value to a string explicitly.

javascript

Copy code

```
var num = 42;
var strNum = String(num); // Explicit conversion to string
console.log(strNum); // "42"
```

2. Number Conversion:

- `Number(value)` can be used to convert a value to a number explicitly.

javascript

Copy code

```
var strNum = "42";
var num = Number(strNum); // Explicit conversion to number
console.log(num); // 42
```

3. Boolean Conversion:

- `Boolean(value)` can be used to convert a value to a boolean explicitly.

javascript

Copy code

```
var nonEmptyString = "Hello";
var boolValue = Boolean(nonEmptyString); // Explicit conversion to boolean
console.log(boolValue); // true
```



4. Truthy and Falsy Values:

- In JavaScript, certain values are considered truthy or falsy in boolean contexts.
 - Falsy values: false, 0, "" (empty string), null, undefined, and NaN.
 - Truthy values: All other values, including non-empty strings, numbers other than 0, and objects.